

Obstacles for a Llama

Lama And platosi (And tog'larida joylashgan tekis yer hududi) bo'ylab sayohat qilishni xohlaydi. Unda $N \times M$ kvadrat kataklardan iborat to'r(grid) ko'rinishidagi plato xaritasi mavjud. Xaritaning satrlari yuqoridan pastgacha 0 dan $N - 1$ gacha, ustunlar esa chapdan o'ngga 0 dan $M - 1$ gacha raqamlangan. Xaritaning i - qatori va j - ustunidagi katak ($0 \leq i < N, 0 \leq j < M$) (i, j) bilan belgilanadi.

Lama platoning iqlimini o'rganib chiqdi va xaritaning har bir qatoridagi barcha kataklar bir xil **harorat**ga va xaritaning har bir ustunidagi barcha kataklar bir xil **namlik**ka ega ekanligini aniqladi. Lama sizga mos ravishda N va M uzunlikdagi butun sonlardan iborat T va H massivlarini berdi. Bu yerda $T[i]$ ($0 \leq i < N$) i - qatordagi kataklarning haroratini, $H[j]$ ($0 \leq j < M$) j - ustundagi kataklarning namligini bildiradi.

Lama, shuningdek, plato florasi(baland tekisliklarda o'sadigan o'simliklar)ni o'rganib chiqdi va (i, j) katagi agar $T[i] > H[j]$ shartni qanoatlantirsa bu katak rasmiy ravishda **unumsiz** (o'simlik o'smaydigan) katak ekanligini payqadi.

Lama plato bo'ylab faqat **yaroqli yo'l** bo'ylab sayohat qilishi mumkin. Yaroqli yo'l - bu quyidagi shartlarni qanoatlantiradigan aniq kataklar ketma-ketligi:

- Yo'ldagi ketma-ket kelgan kataklarning har bir jufti **umumiy tomonga ega**.
- Yo'ldagi barcha kataklar **unumsiz**.

Sizning vazifangiz Q ta so'rovga javob berishdir. Har bir so'rov uchun sizga to'rtta butun son beriladi: L, R, S va D . To'g'ri yo'l mavjudligini aniqlashingiz kerak, shunday qilib:

- Yo'l $(0, S)$ katakdan boshlanib, $(0, D)$ katakda tugaydi.
- Yo'lning barcha kataklari L dan R gacha bo'lgan ustunlar oralig'ida joylashgan.

$(0, S)$ va $(0, D)$ **unumsiz** ekanligi kafolatlangan.

Implement tafsilotlari

Siz implement qilishingiz kerak bo'lgan birinchi protsedura:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : har bir satrdagi haroratni ko'rsatuvchi N uzunlikdagi massiv.

- H : har bir ustundagi namlikni ko'rsatuvchi M uzunlikdagi massiv.
- Har bir test case uchun ushbu protsedura `can_reach` ga murojaat qilishdan oldin aynan bir marta chaqiriladi.

Siz implement qilishingiz kerak bo'lgan ikkinchi protsedura:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : so'rovni ifodalovchi butun sonlar.
- Ushbu protsedura har bir test case uchun Q marta chaqiriladi.

Agar $[L, R]$ oraliqdagi ustunlardan foydalangan holda $(0, S)$ katakdan $(0, D)$ katakgacha yaroqli yo'l mavjud bo'lsa, bu protsedura `true` qaytarishi kerak, aks holda `false` qaytarishi kerak.

Cheklovlar

- $1 \leq N, M, Q \leq 200\,000$
- Har bir i ($0 \leq i < N$) uchun $0 \leq T[i] \leq 10^9$.
- Har bir j ($0 \leq j < M$) uchun $0 \leq H[j] \leq 10^9$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- $(0, S)$ va $(0, D)$ kataklar unumsiz.

Subtask lar

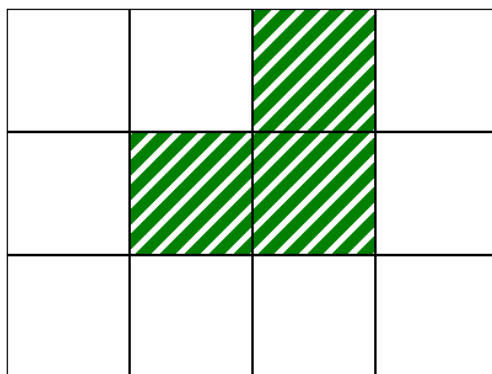
Subtask	Ball	Qo'shimcha Cheklovlar
1	10	Har bir so'rov uchun $L = 0, R = M - 1, N = 1$.
2	14	Har bir so'rov uchun $L = 0, R = M - 1$. Har bir i ($1 \leq i < N$) uchun $T[i - 1] \leq T[i]$.
3	13	Har bir so'rov uchun $L = 0, R = M - 1$. $N = 3$ va $T = [2, 1, 3]$.
4	21	Har bir so'rov uchun $L = 0, R = M - 1$. $Q \leq 10$.
5	25	Har bir so'rov uchun $L = 0, R = M - 1$.
6	17	Qo'shimcha cheklovlarsiz.

Misol

Quyidagi chaqiruvni ko'rib chiqing:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

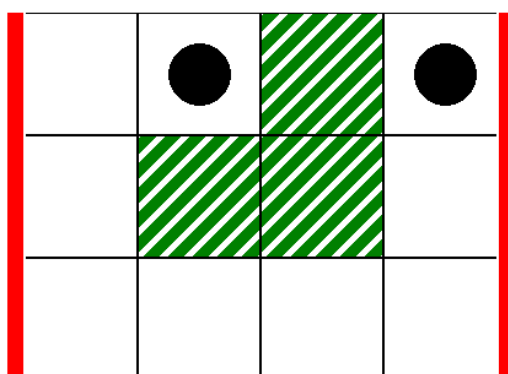
Bu yerda unumsiz kataklar oq katak bilan ifodalangan quyidagi rasmdagi xaritaga mos keladi.



Birinchi so'rov sifatida quyidagi chaqiruvni ko'rib chiqing:

```
can_reach(0, 3, 1, 3)
```

Bu quyidagi rasmdagi stsenariyga mos keladi, bu yerda qalin vertikal chiziqlar $L = 0$ dan $R = 3$ gacha bo'lgan ustunlar oralig'ini, qora aylanalar esa boshlang'ich va tugatish katakchalarini ifodalaydi:



Bu holda Lama $(0, 1)$ katakdan $(0, 3)$ katakga quyidagi yaroqli yo'l orqali yetib borishi mumkin:

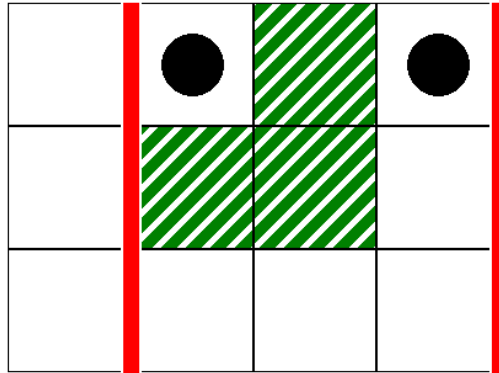
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Shuning uchun, bu chaqiruv `true` qaytarishi kerak.

Ikkinchi so'rov sifatida quyidagi chaqiruvni ko'rib chiqing:

```
can_reach(1, 3, 1, 3)
```

Bu quyidagi rasmdagi stsenariyga mos keladi:



Bunday holda, (0,1) katakdan (0,3) katakga barcha yacheykalar 1 dan 3 gacha bo'lgan ustunlar ichida joylashgan yaroqli yo'l yo'q. Shuning uchun, bu chaqiruv `false` qaytarishi kerak.

Namunaviy greyder

Kiruv formati:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Bu yerda $L[k]$, $R[k]$, $S[k]$ va $D[k]$ ($0 \leq k < Q$) `can_reach` ga har bir chaqiruv uchun parametrlarni ifodalaydi.

Chiqish formati:

```
A[0]
A[1]
...
A[Q-1]
```

Bu yerda $A[k]$ ($0 \leq k < Q$) agar `can_reach(L[k], R[k], S[k], D[k])` chaqiruvi `true` qaytarsa 1 bo'ladi, aks holda 0.